K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology

ENSP465 - New Age Programming Languages Lab

Capstone Project - Phase II

Implementation & Intermediate Demonstration

PROJECT TITLE

Console-Based Student Management System in Go

Student Name	Divyanshi
Roll Number	2301730198
Semester	VII (B.Tech CSE)
Academic Year	2026–2027
Section	C
Phase I Submission Date	24 October 2026
Phase II Submission	November 2026
Faculty Name	Dr. Hemraj Kumawat
Course Code & Name	ENSP465 - New Age Programming Languages Lab

Contents

Contents 2

1. Introduction and Phase I Recap 4

1.1 Purpose of this document 4

1.2 Recap of the proposed system 4

1.3 Scope of Phase II 4

2. Progress Against the Work Plan 5

2.1 Milestone status 5

2.2 Objective status 5

3. Design Refinements since Phase I 6

4. Implemented System Architecture 7

4.1 Layered architecture 7

4.2 Package responsibilities 7

4.3 Project structure 8

4.4 Technology and Go features used 8

5. Module-wise Implementation 9

5.1 Student model 9

5.2 Validation (validator.go) 9

5.3 Result calculation (calc.go) 9

5.4 Service layer 10

5.5 Storage layer 11

5.6 Console interface (ui) 11

5.7 End-to-end flow of “Add Student” 11

6. Data Persistence and File Format 13

Why JSON? 13

Safety guarantees 13

7. Intermediate Demonstration - Console Screens 14

7.1 Main menu 14

7.2 Add student 14

7.3 Input validation - age and marks out of range 14

7.4 Duplicate ID and cancel 15

7.5 View all students 15

7.6 Search by ID and by name 15

7.7 Search with no result and unknown ID . . . 16
7.8 Update student . . . 16
7.9 Delete student with confirmation . . . 17
7.10 Invalid menu choice and unknown ID on update . . . 17
7.11 Exit and persistence . . . 18
8. Verification - Manual Test Cases . . . 19
9. Design Challenges and How They Were Handled . . . 20
10. Limitations and Plan for Phase III . . . 20
10.1 Current limitations . . . 20
10.2 Phase III work (milestones 5 and 6) . . . 20
11. How to Build and Run . . . 21
12. Conclusion . . . 21
13. References . . . 21

1. Introduction and Phase I Recap

1.1 Purpose of this document

This document is the Phase II deliverable of the ENSP465 Capstone Project. Phase I presented the problem, the choice of Go as the implementation language, the objectives, and the system design. Phase II covers **implementation and the intermediate demonstration**: it explains how the design was turned into a working console application, which design decisions were refined along the way, and it shows the application running through real captured console output for every feature.

1.2 Recap of the proposed system

Small institutes and coaching centres often store student data in paper registers or spreadsheets, where totals and grades are computed by hand, duplicate records creep in, and searching is slow. The project is a lightweight, offline **console application written in Go** that manages student records through a simple menu and keeps them saved between runs. It needs no server, database or network connection.

Each student record holds an ID, name, age, course and marks in five subjects. The program calculates the total, percentage and grade automatically and stores everything in a local JSON file.

1.3 Scope of Phase II

According to the Phase I work plan, Phase II covers milestones 2 to 4: the core model, CRUD operations with persistence, and the complete menu-driven console interface with an intermediate demonstration. The following are delivered in this phase:

- The five CRUD operations - add, view, search, update and delete - available through the menu.
- Automatic calculation of total marks, percentage and grade.
- JSON persistence: records are loaded at start-up and saved after every change.
- Complete input validation with friendly, specific error messages.
- A layered package structure (ui, service, student, store) as designed in Phase I, with refinements described in Section 3.

Out of scope for Phase II (and for the project as a whole) remain the items listed in Phase I: no GUI or web front end, no login or roles, no relational database or cloud storage, no attendance, fee or timetable modules, and no multi-user concurrent access.

2. Progress Against the Work Plan

2.1 Milestone status

The table below compares the Phase I work plan with the current status. Milestones 2, 3 and 4 form Phase II; milestones 5 and 6 belong to Phase III.

No.	Milestone	Deliverables (from Phase I plan)	Status
1	Phase I submission	Proposal and system design document	Completed (24 Oct 2026)
2	Setup and core model	Go module, project structure, Student struct, calculator and validator	Completed
3	CRUD and persistence	Service layer, JSON storage, add / view / search / update / delete working	Completed
4	Phase II: console UI and demo	Complete menu-driven interface, intermediate demonstration	Completed - demonstrated in Section 7
5	Testing and refinement	Unit and manual test cases, bug fixes, edge-case handling, clean-up	Planned - Phase III
6	Phase III: final deliverable	Final source code, README, user guide, test report, viva preparation	Planned - Phase III

2.2 Objective status

Phase I defined five measurable objectives. The table shows where each stands at the end of Phase II.

#	Objective (Phase I)	Status after Phase II	Evidence
1	Five CRUD operations through a menu-driven console interface	Achieved	Sections 7.2 – 7.9 show add, view, search, update and delete working end-to-end
2	Automatic total, percentage and grade	Achieved	Worked examples in Sections 5.3 and 7 (e.g. 413/500 = 82.60 % = grade A); a larger test set of at least 10 students is part of Phase III testing
3	Persist records in JSON; restore after restart	Achieved	Section 7.11 shows the records and the saved students.json after a restart
4	Validate all input; invalid data never corrupts stored records	Achieved	Sections 7.3, 7.4, 7.7, 7.10 show rejected input with clear messages
5	Unit tests covering at least 70 % of calculator, validator and service code	Planned - Phase III	Section 10 lists the planned test work

3. Design Refinements since Phase I

While implementing the design, several details were refined. None of them changes the overall layered architecture or the scope; they make the program safer, simpler to test, or easier to use. The table summarises each change and the reason for it.

Area	Phase I design	Phase II implementation	Reason
Packages	Separate packages: ui, service, store, calc, validator	Four packages: ui, service, store, student. The student package holds the Student struct plus validator.go and calc.go	Validation and calculation rules both describe a student, so keeping them next to the struct is more cohesive and avoids extra cross-package dependencies
Saving data	Save the collection after each modification	Each change is saved first and only then applied in memory; if saving fails, the in-memory data stays unchanged	Memory and file can never disagree after a failed write
Console I/O	Read input with bufio.Scanner	ui works with injected io.Reader / io.Writer streams	The same code can run on the real console or on scripted input, which makes the interface testable
Marks limit	“0 to maximum”	Five subjects, each 0–100; total is out of 500	Fixes the maximum so percentage = total ÷ 500 × 100
Numeric validation	Range check only	Range check plus rejection of NaN and Inf values	Special floating-point values can otherwise pass a naive range comparison or break calculations
Navigation	Not specified	Typing **cancel** at any prompt returns to the main menu	The user is never trapped part-way through a long form
Update flow	“Update any field”	Current record is displayed; the old value appears in brackets; pressing Enter keeps it	Only the changed fields need to be typed
Delete flow	Delete after confirmation	Record is shown, then a y/n confirmation is asked	The user can verify the right record before it is removed
Output	Aligned tables	Tables produced with text/tabwriter; ✓ and ✗ markers on messages	Consistent alignment and easily readable success / error feedback

4. Implemented System Architecture

4.1 Layered architecture

The application follows the layered design proposed in Phase I. Each layer has exactly one responsibility and only talks to the layer directly below it. The presentation layer never touches the file, and the storage layer knows nothing about menus or validation rules.

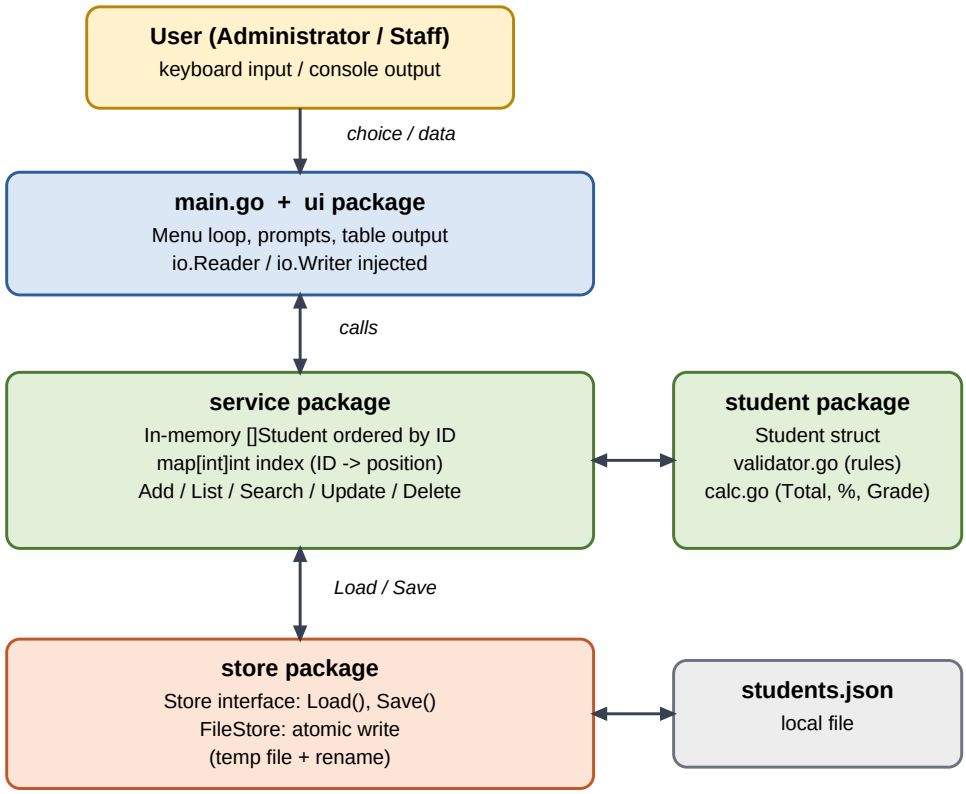


Figure 1: Implemented architecture of the Student Management System

The call chain is **main.go** → **ui** → **service** → **store** → **students.json**. The service layer also uses the student package for the data model, validation and result calculation.

4.2 Package responsibilities

Package	Responsibility
ui	Runs the menu loop, shows prompts, reads answers, re-asks on invalid input, prints aligned tables and messages. It contains no business logic and uses injected io.Reader / io.Writer streams.
service	Holds the in-memory []Student slice (kept ordered by ID) and a map[int]int index from ID to position, giving O(1) lookup and duplicate detection. Implements Add, List, SearchByID, SearchByName, Update and Delete. Every change is saved first and applied in memory only if saving succeeds.
student	Defines the Student struct (with JSON tags). validator.go contains field-level and whole-record validation rules that return sentinel errors. calc.go provides Total, Percentage, Grade and Compute; the grading bands live in a single table so they are easy to change.

Package	Responsibility
store	Defines the Store interface (Load, Save) and the FileStore implementation, which writes atomically using a temporary file followed by a rename.

4.3 Project structure

```
project folder/
├── main.go                 entry point: wires store, service and ui together
├── students.json           data file (created / updated by the program)
├── ui/                     menu loop, prompts, table output
├── service/                in-memory collection, index, CRUD rules
├── student/
│   ├── (Student struct)    typed fields with JSON tags
│   ├── validator.go        field-level and record-level rules
│   └── calc.go             Total, Percentage, Grade, Compute
└── store/                  Store interface and atomic FileStore
```

Figure 2: Package layout of the project

4.4 Technology and Go features used

Only the Go standard library is used (Go 1.21 or newer); no third-party dependency is required.

Standard library package	Used for
fmt, bufio, os	Console prompts, reading user input line by line, formatted output, file access
encoding/json	Converting the []Student collection to and from students.json (indented output)
strconv, strings	Converting text to numbers; trimming input; case-insensitive partial name matching
sort	Keeping and displaying records in ascending ID order
errors	Sentinel errors and error checks with errors.Is
text/tabwriter	Aligned columns in all table output

The following Go language features are demonstrated in the code:

- **Structs with JSON tags** - Student maps directly to the file format.
- **Slices and maps** - a slice preserves display order; a map gives constant-time lookup.
- **Packages** - ui, service, student and store each expose a small public API.
- **Interfaces** - the Store interface lets the service work with any storage backend, including a test double.
- **Custom and wrapped errors** - sentinel errors wrapped with %w, tested using errors.Is.
- **defer** - guarantees that files are closed and cleanup happens on every code path.

5. Module-wise Implementation

5.1 Student model

A student record is represented by one struct. Its fields are written to and read from the JSON file using struct tags, so no manual conversion code is needed. The derived values (total, percentage, grade) are stored alongside the raw data so that the file is complete and readable on its own.

Field	JSON key	Meaning
ID	id	Unique, positive whole number identifying the student
Name	name	Student's full name (not empty)
Age	age	Age in years, between 5 and 100
Course	course	Course the student is enrolled in (not empty)
Marks	marks	List of five subject marks, each 0–100
Total	total	Sum of the five marks (calculated)
Percentage	percentage	Total ÷ 500 × 100 (calculated)
Grade	grade	Letter grade from the grading table (calculated)

5.2 Validation (validator.go)

Validation lives in one place, so the same rules apply no matter where data comes from. There are two kinds of check: **field-level** checks, used by the console while a single answer is being typed, and **whole-record** checks, used by the service before a record is stored. Both return sentinel errors that the interface turns into friendly messages.

Field	Rule	Message shown to the user
ID	Must be a whole number; positive; not already used by another student	"ID must be a whole number" "a student with this ID already exists"
Name	Must not be empty	Prompt is repeated with an error message
Age	Whole number from 5 to 100	"age must be between 5 and 100"
Course	Must not be empty	Prompt is repeated with an error message
Marks	Each of the five marks from 0 to 100; NaN and Inf are rejected	"invalid marks: marks must be between 0 and 100"

When an answer is invalid, the program prints a ✗ message and asks the **same question again**. Invalid data therefore never reaches the service layer or the stored records.

5.3 Result calculation (calc.go)

The calculator is a set of small, pure functions: **Total** adds the marks; **Percentage** computes total ÷ 500 × 100; **Grade** maps the percentage to a letter; **Compute** fills in all three on a student record. Because the functions have no side effects, they are easy to test with fixed inputs. Compute is called whenever a student is added or marks are changed, so the stored result can never be out of date.

Percentage	Grade	Meaning
90 and above	O	Outstanding
80 to 89.99	A	Excellent
70 to 79.99	B	Good
60 to 69.99	C	Satisfactory
50 to 59.99	D	Pass
Below 50	F	Fail

The bands are stored in a single table inside calc.go. Changing the grading scheme means editing that table only; no other code changes.

Worked example. Student 101, Aman Verma, has marks 78, 85, 90, 72 and 88:

Step	Calculation	Result
Total	78 + 85 + 90 + 72 + 88	413 out of 500
Percentage	413 ÷ 500 × 100	82.60 %
Grade	82.60 lies in the 80 to 89.99 band	A

5.4 Service layer

The service layer implements the business rules. It keeps two structures in memory:

- a **slice** of students ordered by ID, used for listing and for writing the file; and
- a **map** from ID to slice position, used for instant lookup and for rejecting duplicate IDs.

Behaviour of each operation:

Operation	What it does	Possible errors
Add	Validates the record, checks the ID is unused, computes total / percentage / grade, saves, then inserts it in ID order and updates the index	duplicate ID; invalid field; save failure
List	Returns all students sorted by ID	-
SearchByID	Looks the ID up in the map	student not found
SearchByName	Case-insensitive partial match on the name; may return several students	none (an empty result is reported by the UI)
Update	Applies new values, re-validates, recalculates results, saves, then replaces the record	student not found; invalid field; save failure
Delete	Removes the record, saves, then rebuilds the index so positions remain correct	student not found; save failure

Save first, then apply. Every modifying operation builds the new collection, asks the store to save it, and updates the in-memory slice and index only if the save succeeded. If the disk write fails, the in-memory data remains exactly as it was, so memory and file never contradict each other.

Wrapped errors. Errors such as “student not found” are wrapped with context using %w, producing messages like `ID 999: student not found`. Callers can still test the underlying reason with errors.Is.

5.5 Storage layer

The storage layer is hidden behind a two-method **Store interface**: Load and Save. The service depends only on this interface, so another backend or a mock can replace the file store without changing the service. The real implementation, FileStore, behaves as follows:

- **Load** reads students.json at start-up. If the file does not exist, the program starts with an empty collection.
- **Save** writes the complete collection as indented JSON.

Saving is **atomic**: the data is first written completely to a temporary file in the same folder, and only then is the temporary file renamed over students.json. A rename is a single operation on the file system, so the data file is always either the full old version or the full new version. A crash or power failure during saving cannot leave a half-written file. The defer statement ensures files are closed on every path.

5.6 Console interface (ui)

The ui package contains the menu loop and one handler per menu option. Its main characteristics are:

- **Re-prompt on error** - an invalid answer prints a ✗ message and the same prompt appears again.
- **cancel** - typing cancel at any prompt prints “Cancelled.” and returns to the main menu.
- **Update with defaults** - the current value is shown in square brackets; pressing Enter keeps it.
- **Delete confirmation** - the record is displayed and a y/n answer is required; n aborts the deletion.
- **Aligned tables** - produced with text/tabwriter for all views, searches and confirmations.
- **Injected streams** - the ui reads from an io.Reader and writes to an io.Writer instead of using os.Stdin and os.Stdout directly. In normal use these are the console; for testing they can be a scripted input and a captured output.

5.7 End-to-end flow of “Add Student”

The sequence diagram in Phase I (Figure 3) is realised as follows. Steps 1–3 happen in the ui, step 4 in the service, and steps 5–8 in the student and store packages.

Step	Component	Action
1	ui	User chooses option 1; the ui prints the “Add Student” heading and asks for each field in turn
2	ui + student/validator	Each answer is checked as soon as it is typed; if invalid, an error is printed and the same prompt repeats; “cancel” returns to the menu
3	ui	When all fields are valid, the ui hands the complete record to the service
4	service	Re-checks the whole record and confirms the ID is not already in the index (duplicate → error)

Step	Component	Action
5	student/calc	Computes total, percentage and grade
6	store	Saves the new collection to students.json using the temp-file-and-rename method
7	service	Only after a successful save: inserts the record in ID order and updates the index
8	ui	Prints “✓ Student added successfully.” followed by the new record as a table row

6. Data Persistence and File Format

All data is kept in one human-readable file, **students.json**, in the program's working folder. The file holds a JSON array with one object per student. It is loaded when the program starts and rewritten after every successful change, so closing the program at any time never loses an operation that was already confirmed.

The file below is the one produced by the demonstration in Section 7. Marks are shown compactly here; on disk Go's indented output places each mark on its own line.


```
[
  {
    "id": 101,
    "name": "Aman Verma",
    "age": 21,
    "course": "B.Tech CSE",
    "marks": [78, 85, 90, 72, 95],
    "total": 420,
    "percentage": 84,
    "grade": "A"
  },
  {
    "id": 103,
    "name": "Aman Gupta",
    "age": 22,
    "course": "B.Sc",
    "marks": [55, 48, 62, 51, 58],
    "total": 274,
    "percentage": 54.8,
    "grade": "D"
  }
]
```


Figure 3: Contents of students.json after the demonstration session

Why JSON?

- It maps directly onto a Go struct through struct tags, so the standard library does all the work.
- It is readable and editable in any text editor, which helps debugging and manual inspection.
- It needs no installation or administration, unlike a database server.

Safety guarantees

- **Atomic write** - temp file + rename means the file is never half-written.
- **Save before memory update** - a failed save leaves the in-memory data unchanged.
- **Validated data only** - only records that passed every rule can ever be written to the file.

7. Intermediate Demonstration - Console Screens

Every screen in this section is **real output captured from the running program**. The text typed after each prompt is what the user entered. The demonstration follows one continuous session: three students are added, searched, updated and one is deleted, and the program is then restarted to prove persistence. Typing `cancel` at any prompt returns to the main menu.

7.1 Main menu

On start-up the program loads students.json and shows the main menu with six options. The last line reminds the user about the cancel command.

```
========== STUDENT MANAGEMENT ==========
 1. Add student
 2. View all students
 3. Search student
 4. Update student
 5. Delete student
 6. Exit
========================================
 (type 'cancel' at any prompt to go back)
Enter choice (1-6):
```

Figure 4: Main menu

7.2 Add student

The first attempt at the ID is “abc”, which is not a whole number, so the program prints a ✗ message and asks again. After a valid ID the remaining fields are entered. Once the last mark is accepted, the student is saved and shown in a table with the automatically calculated total, percentage and grade (413/500, 82.60 %, grade A).

```
Enter choice (1-6): 1

--- Add Student ---
Student ID: abc
  ✗ ID must be a whole number
Student ID: 101
Name: Aman Verma
Age: 21
Course: B.Tech CSE
Marks for Subject 1 (0-100): 78
Marks for Subject 2 (0-100): 85
Marks for Subject 3 (0-100): 90
Marks for Subject 4 (0-100): 72
Marks for Subject 5 (0-100): 88
  ✓ Student added successfully.
ID   NAME        AGE  COURSE      MARKS           TOTAL    PERCENT  GRADE
--   ----        ---  ------      -----           -----    -------  -----
101  Aman Verma  21   B.Tech CSE  78 85 90 72 88  413/500  82.60%   A
```

Figure 5: Adding a student (with one invalid ID rejected)

7.3 Input validation - age and marks out of range

For the second student, an age of 3 is rejected because the allowed range is 5–100, and a mark of 105 is rejected because marks must be between 0 and 100. In both cases the same prompt is shown again and the valid value is accepted. The record is saved only when every field is valid: 471/500 = 94.20 %, grade O.

```
Enter choice (1-6): 1

--- Add Student ---
Student ID: 102
Name: Riya Sharma
Age: 3
  ✗ age must be between 5 and 100
Age: 20
Course: BCA
Marks for Subject 1 (0-100): 95
Marks for Subject 2 (0-100): 92
Marks for Subject 3 (0-100): 105
  ✗ invalid marks: marks must be between 0 and 100
Marks for Subject 3 (0-100): 98
Marks for Subject 4 (0-100): 90
Marks for Subject 5 (0-100): 96
  ✓ Student added successfully.
ID   NAME         AGE  COURSE  MARKS           TOTAL    PERCENT  GRADE
--   ----         ---  ------  -----           -----    -------  -----
102  Riya Sharma  20   BCA     95 92 98 90 96  471/500  94.20%   O
```

Figure 6: Age and marks validation

7.4 Duplicate ID and cancel

Entering ID 101 again is refused because that ID already exists (detected through the map index). Typing `cancel` at the next prompt abandons the operation and returns to the menu without changing any data.

```
Enter choice (1-6): 1

--- Add Student ---
Student ID: 101
  ✗ a student with this ID already exists
Student ID: cancel
  Cancelled.
```

Figure 7: Duplicate ID rejected; cancel returns to the menu

7.5 View all students

Option 2 lists every student in ascending ID order. The header shows the number of records. A third student (ID 103, Aman Gupta, 54.80 %, grade D) has been added earlier in the session in the same way.

```
Enter choice (1-6): 2

--- All Students (3) ---
ID   NAME         AGE  COURSE      MARKS           TOTAL    PERCENT  GRADE
--   ----         ---  ------      -----           -----    -------  -----
101  Aman Verma   21   B.Tech CSE  78 85 90 72 88  413/500  82.60%   A
102  Riya Sharma  20   BCA         95 92 98 90 96  471/500  94.20%   O
103  Aman Gupta   22   B.Sc        55 48 62 51 58  274/500  54.80%   D
```

Figure 8: Table of all students

7.6 Search by ID and by name

Search offers two modes. Searching by ID returns exactly one record. Searching by name is **partial and case-insensitive**: the keyword “aman” matches both “Aman Verma” and “Aman Gupta”, and the program reports the number of matches.

```
Enter choice (1-6): 3

--- Search ---
 1. By ID
 2. By name (partial match)
Choice: 1
Student ID: 101
ID   NAME        AGE  COURSE      MARKS           TOTAL    PERCENT  GRADE
--   ----        ---  ------      -----           -----    -------  -----
101  Aman Verma  21   B.Tech CSE  78 85 90 72 88  413/500  82.60%   A
Enter choice (1-6): 3

--- Search ---
 1. By ID
 2. By name (partial match)
Choice: 2
Name contains: aman
  2 match(es):
ID   NAME        AGE  COURSE      MARKS           TOTAL    PERCENT  GRADE
--   ----        ---  ------      -----           -----    -------  -----
101  Aman Verma  21   B.Tech CSE  78 85 90 72 88  413/500  82.60%   A
103  Aman Gupta  22   B.Sc        55 48 62 51 58  274/500  54.80%   D
```

Figure 9: Search by ID and by partial name

7.7 Search with no result and unknown ID

A name that matches nobody gives a plain “No students match” message. An unknown ID produces an error built from the wrapped ErrNotFound error: the message includes the ID that was searched for.

```
Enter choice (1-6): 3

--- Search ---
 1. By ID
 2. By name (partial match)
Choice: 2
Name contains: zzz
  No students match "zzz".
Enter choice (1-6): 3

--- Search ---
 1. By ID
 2. By name (partial match)
Choice: 1
Student ID: 999
  ✗ Error: ID 999: student not found
```

Figure 10: Empty result and unknown ID

7.8 Update student

The update screen first shows the current record. For each field the current value appears in square brackets, and pressing Enter keeps it. Here only Subject 5 is changed from 88 to 95. The program recalculates the results automatically: total 413 → 420, percentage 82.60 % → 84.00 %, grade remains A.

```
Enter choice (1-6): 4

--- Update Student ---
Student ID to update: 101
  Current record (press Enter to keep a value):
ID   NAME        AGE  COURSE      MARKS           TOTAL    PERCENT  GRADE
--   ----        ---  ------      -----           -----    -------  -----
101  Aman Verma  21   B.Tech CSE  78 85 90 72 88  413/500  82.60%   A
Name [Aman Verma]:
Age [21]:
Course [B.Tech CSE]:
Marks for Subject 1 (0-100) [78]:
Marks for Subject 2 (0-100) [85]:
Marks for Subject 3 (0-100) [90]:
Marks for Subject 4 (0-100) [72]:
Marks for Subject 5 (0-100) [88]: 95
  ✓ Student updated successfully.
ID   NAME        AGE  COURSE      MARKS           TOTAL    PERCENT  GRADE
--   ----        ---  ------      -----           -----    -------  -----
101  Aman Verma  21   B.Tech CSE  78 85 90 72 95  420/500  84.00%   A
```

Figure 11: Updating one field while keeping the others

7.9 Delete student with confirmation

Deleting first displays the record and asks for confirmation. The first attempt answers **n**, so the deletion is aborted and the record stays. The second attempt answers **y**, the record is removed, and the following “View all” shows two remaining students.

```
Enter choice (1-6): 5

--- Delete Student ---
Student ID to delete: 102
ID   NAME         AGE  COURSE  MARKS           TOTAL    PERCENT  GRADE
--   ----         ---  ------  -----           -----    -------  -----
102  Riya Sharma  20   BCA     95 92 98 90 96  471/500  94.20%   O
Delete this student? (y/n): n
  Deletion aborted.
Enter choice (1-6): 5

--- Delete Student ---
Student ID to delete: 102
ID   NAME         AGE  COURSE  MARKS           TOTAL    PERCENT  GRADE
--   ----         ---  ------  -----           -----    -------  -----
102  Riya Sharma  20   BCA     95 92 98 90 96  471/500  94.20%   O
Delete this student? (y/n): y
  ✓ Student deleted.
Enter choice (1-6): 2

--- All Students (2) ---
ID   NAME        AGE  COURSE      MARKS           TOTAL    PERCENT  GRADE
--   ----        ---  ------      -----           -----    -------  -----
101  Aman Verma  21   B.Tech CSE  78 85 90 72 95  420/500  84.00%   A
103  Aman Gupta  22   B.Sc        55 48 62 51 58  274/500  54.80%   D
```

Figure 12: Delete aborted with n, then confirmed with y

7.10 Invalid menu choice and unknown ID on update

Anything other than 1–6 at the main menu is rejected with a message explaining the valid range. Trying to update an ID that does not exist produces the same “student not found” error as searching.

```
Enter choice (1-6): 9
  ✗ Invalid choice. Please enter a number from 1 to 6.
Enter choice (1-6): 4

--- Update Student ---
Student ID to update: 555
  ✗ Error: ID 555: student not found
```

Figure 13: Invalid menu choice and unknown ID on update

7.11 Exit and persistence

Choosing 6 exits the program and confirms that everything has been saved. After starting the program again, option 2 shows exactly the same two records, which were loaded from students.json (its contents are shown in Figure 3).

```
Enter choice (1-6): 6
Goodbye! All changes are saved.
```

Figure 14: Exiting the program

```
Enter choice (1-6): 2

--- All Students (2) ---
ID   NAME        AGE  COURSE      MARKS           TOTAL    PERCENT  GRADE
--   ----        ---  ------      -----           -----    -------  -----
101  Aman Verma  21   B.Tech CSE  78 85 90 72 95  420/500  84.00%   A
103  Aman Gupta  22   B.Sc        55 48 62 51 58  274/500  54.80%   D
```

Figure 15: After restart - the saved records are restored

8. Verification - Manual Test Cases

The table lists the test cases exercised during the demonstration. The expected result is what the specification requires; the observed result is taken from the captured output in Section 7. All cases passed. Formal automated unit tests (table-driven tests for grade boundaries and validation rules, and service tests using a mock Store) and a coverage report are scheduled for Phase III.

#	Test case	Input	Expected result	Observed	Result
1	Add a valid student	ID 101, Aman Verma, 21, B.Tech CSE, marks 78 85 90 72 88	Saved; total 413, 82.60 %, grade A	As expected (Fig. 5)	Pass
2	Non-numeric ID	abc	Rejected; ID asked again	“ID must be a whole number”	Pass
3	Age below minimum	Age 3	Rejected; age asked again	“age must be between 5 and 100”	Pass
4	Mark above maximum	Mark 105	Rejected; mark asked again	“marks must be between 0 and 100”	Pass
5	Grade O boundary region	Marks 95 92 98 90 96	471/500, 94.20 %, grade O	As expected (Fig. 6)	Pass
6	Duplicate ID	ID 101 again	Rejected	“a student with this ID already exists”	Pass
7	Cancel during add	cancel	Return to menu, no change	“Cancelled.”	Pass
8	View all, sorted	Option 2	Rows ordered by ID	101, 102, 103	Pass
9	Search by ID	101	Exactly one record	As expected (Fig. 9)	Pass
10	Partial, case-insensitive name search	“aman”	Both Aman records	2 matches	Pass
11	Name with no match	“zzz”	Friendly “no match” message	As expected	Pass
12	Unknown ID on search	999	Not-found error	“ID 999: student not found”	Pass
13	Update one field, keep others	Enter ×7, then 95 for Subject 5	Results recalculated: 420, 84.00 %, A	As expected (Fig. 11)	Pass
14	Unknown ID on update	555	Not-found error	“ID 555: student not found”	Pass
15	Delete, answer n	ID 102, n	Record kept	“Deletion aborted.”	Pass
16	Delete, answer y	ID 102, y	Record removed	“Student deleted.”; 2 records remain	Pass
17	Invalid menu choice	9	Message, menu shown again	“Please enter a number from 1 to 6”	Pass
18	Persistence after restart	Exit, restart, option 2	Same records restored	Identical records (Fig. 15)	Pass

9. Design Challenges and How They Were Handled

Challenge	Solution in the implementation
A crash or power failure while writing could leave a corrupted data file	Atomic save: write to a temporary file, then rename over students.json. The file is always a complete old or new version.
A failed save could leave memory and file disagreeing	The service saves first and updates memory only after success.
Slow duplicate detection and lookup on a growing list	A map[int]int index from ID to slice position gives O(1) lookup; the slice keeps display order.
Bad input must never reach stored data	Field-level validation at the prompt (with re-ask) plus whole-record validation in the service; NaN and Inf are explicitly rejected.
Users could get stuck in the middle of a long form	A global cancel command returns to the menu from any prompt.
Retyping every field for a small correction is tedious	Update shows current values; pressing Enter keeps them.
Accidental deletion	The record is shown and a y/n confirmation is required.
Console programs are hard to test	ui uses injected io.Reader / io.Writer streams, so scripted input and captured output can be used in tests.
Results stale after marks change	Compute runs on every add and update, so total, percentage and grade always match the marks.

10. Limitations and Plan for Phase III

10.1 Current limitations

- Single local user only; there is no locking for several programs using the same file.
- The whole collection is rewritten on every change. This is fine for hundreds of students but would not suit very large data sets.
- The number of subjects is fixed at five and each subject is out of 100.
- No pagination: “View all” prints every record at once.

10.2 Phase III work (milestones 5 and 6)

Task	Detail	Target
Unit tests	Table-driven tests for calculator (grade boundaries), validator rules and service operations, using a mock Store	≥ 70 % coverage of calculator, validator and service code
Calculation check	Compare program results with hand-computed results for a test set of at least 10 students	100 % agreement
Persistence check	Verify that every saved record is restored correctly after restart, including a missing and an empty data file	100 % restored
Edge cases and clean-up	Run gofmt and go vet, remove duplicated code, review error messages	1st week of Dec 2026
Documentation and viva	README, user guide, test report; preparation to explain the architecture, Go features and design decisions	2nd week of Dec 2026

11. How to Build and Run

Requirements: Go 1.21 or newer installed. No other software or internet connection is needed.

Step	Command	Result
1	Open a terminal in the project folder (the one containing main.go)	-
2	`go run .`	Compiles and starts the program; the main menu appears
3	Use the menu; type `cancel` at any prompt to go back	Records are saved to students.json after every change
4	Choose 6 to exit	“Goodbye! All changes are saved.”

On the first run students.json does not exist, so the program starts with an empty collection and creates the file when the first student is saved. To start fresh, close the program and delete students.json.

12. Conclusion

Phase II turned the Phase I design into a working application. The program provides all five CRUD operations through a friendly console menu, calculates total, percentage and grade automatically, validates every input, and persists data safely in a JSON file using atomic writes. The layered structure - ui, service, student and store - keeps responsibilities separate, which made the refinements in Section 3 possible without changing the architecture. The captured console output in Section 7 demonstrates every feature, including error handling and persistence across restarts.

Phase III will add automated unit tests, a coverage report, edge-case checks, the README and user guide, and viva preparation, completing the project as planned in Phase I.

13. References

1. The Go Authors, “The Go Programming Language Documentation,” go.dev. [Online]. Available: https://go.dev/doc/. Accessed: November 2026.
2. The Go Authors, “Effective Go,” go.dev. [Online]. Available: https://go.dev/doc/effective_go. Accessed: November 2026.
3. The Go Authors, “Package json - encoding/json,” Go Standard Library. [Online]. Available: https://pkg.go.dev/encoding/json. Accessed: November 2026.
4. The Go Authors, “Package tabwriter - text/tabwriter,” Go Standard Library. [Online]. Available: https://pkg.go.dev/text/tabwriter. Accessed: November 2026.
5. The Go Authors, “Package testing,” Go Standard Library. [Online]. Available: https://pkg.go.dev/testing. Accessed: November 2026.
6. A. A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. Boston, MA, USA: Addison-Wesley, 2015.
7. Course material, ENSP465 - New Age Programming Languages Lab, School of Engineering & Technology, K.R. Mangalam University.